

FIAP — Pós-Tech Full-Stack Development

Tech Challenge — Fase 02

BLOG EDUCACIONAL

API REST — Documentação Técnica

Desenvolvido em Node.js, Express e Prisma ORM

Felipe Franco — RM371023

Gabriel Sancinetti — RM372901

Vinicius Godoy — RM372941

São Paulo — 2026

1. Introdução

O **Tech Challenge** é a atividade integradora da Pós-Tech em Full-Stack Development da FIAP e tem como objetivo consolidar, em um único projeto, os conhecimentos adquiridos ao longo da fase do curso. Nesta segunda etapa, a proposta consiste na evolução da aplicação desenvolvida anteriormente, substituindo o back-end implementado em uma plataforma low-code por uma solução construída com tecnologias amplamente utilizadas no mercado de desenvolvimento de software. O desafio exige não apenas a implementação de novas funcionalidades, mas também a adoção de boas práticas de arquitetura, persistência de dados, segurança, testes automatizados e documentação técnica.

Na **Fase 01**, a equipe desenvolveu um **Blog Educacional** utilizando a plataforma OutSystems. A aplicação foi concebida para atender uma necessidade identificada na comunidade: oferecer aos professores da rede pública de ensino uma plataforma simples, centralizada e acessível para publicação de conteúdos educacionais, permitindo que estudantes consultassem materiais de forma organizada. O projeto atingiu seus objetivos funcionais e demonstrou o potencial da solução, servindo como base para sua evolução.

Com a necessidade de tornar a aplicação mais flexível, escalável e aderente às práticas de desenvolvimento profissional, a **Fase 02** propõe a reconstrução completa da camada de back-end utilizando **Node.js** e **Express**, adotando uma arquitetura baseada em **API REST**. Essa mudança representa uma evolução significativa em relação à implementação anterior, permitindo que a aplicação deixe de depender de uma plataforma específica e passe a disponibilizar seus serviços para qualquer cliente capaz de consumir requisições HTTP, como aplicações web, dispositivos móveis ou mesmo o front-end desenvolvido anteriormente em OutSystems.

Além da migração tecnológica, o projeto incorpora uma camada de persistência utilizando **Prisma ORM** sobre um banco de dados relacional **PostgreSQL**, possibilitando o armazenamento permanente das informações de usuários e postagens. Também foram implementados mecanismos de autenticação e autorização baseados em **JSON Web Token (JWT)**, garantindo que apenas usuários autorizados possam realizar operações de criação, edição e exclusão de conteúdos, enquanto as funcionalidades de consulta permanecem acessíveis ao público.

Outro aspecto importante desta fase é a preocupação com a qualidade do software produzido. Para isso, foram adotadas práticas modernas de Engenharia de Software, como organização do código em camadas, separação de responsabilidades, validação de dados, criptografia de senhas, containerização da aplicação com Docker, utilização de controle de versão com Git e desenvolvimento de testes automatizados para validação das principais

funcionalidades da API. Essas decisões contribuem para tornar o sistema mais robusto, reutilizável, de fácil manutenção e preparado para futuras evoluções.

Sob o ponto de vista arquitetural, a aplicação foi estruturada em camadas independentes, separando responsabilidades entre rotas, controladores, serviços, repositórios e acesso ao banco de dados. Essa abordagem reduz o acoplamento entre os componentes do sistema, facilita a realização de testes unitários e de integração e permite que alterações em uma camada tenham impacto mínimo nas demais, seguindo princípios amplamente adotados no desenvolvimento de aplicações corporativas.

O presente documento tem como finalidade descrever detalhadamente todos os aspectos técnicos relacionados ao desenvolvimento da aplicação. São apresentados a arquitetura adotada, as tecnologias utilizadas, o modelo de dados, a estrutura da API, o processo de instalação e execução, os fluxos de utilização, as decisões arquiteturais tomadas durante o desenvolvimento, os desafios enfrentados pela equipe e as perspectivas de evolução do projeto.

Ao final desta documentação, espera-se demonstrar não apenas que a solução atende aos requisitos funcionais estabelecidos para o Tech Challenge da Fase 02, mas também que sua implementação segue boas práticas de desenvolvimento de software, oferecendo uma base sólida para futuras expansões e para sua utilização em um cenário de produção com diferentes aplicações consumidoras da API.

2. Arquitetura do Sistema

A arquitetura do Blog Educacional foi concebida com o objetivo de oferecer uma solução organizada, escalável e de fácil manutenção, seguindo princípios amplamente adotados no desenvolvimento de aplicações back-end modernas. Para isso, foi utilizada uma **arquitetura em camadas (Layered Architecture)**, na qual cada componente possui responsabilidades bem definidas e independentes.

Essa abordagem permite que alterações em uma camada tenham impacto mínimo sobre as demais, facilitando a manutenção do código, a implementação de testes automatizados e a evolução contínua da aplicação. Além disso, a separação de responsabilidades torna o sistema mais modular, permitindo que novas funcionalidades sejam adicionadas sem comprometer a estrutura existente.

A aplicação foi desenvolvida como uma **API REST**, responsável por disponibilizar os recursos do sistema através de endpoints HTTP. Dessa forma, qualquer cliente compatível com o protocolo HTTP — como aplicações web,

dispositivos móveis ou outros sistemas — pode consumir os serviços oferecidos pela API.

A estrutura da aplicação é composta pelas seguintes camadas:

- **Routes:** definem os endpoints disponíveis e direcionam cada requisição para o controller correspondente;
- **Middlewares:** executam validações, autenticação, autorização e demais processamentos antes que a requisição alcance a lógica de negócio;
- **Controllers:** recebem as requisições HTTP, processam seus parâmetros e delegam a execução das regras de negócio para a camada de serviços;
- **Services:** concentram toda a lógica de negócio da aplicação, garantindo que as regras sejam aplicadas de forma centralizada;
- **Repositories:** realizam o acesso ao banco de dados, abstraindo as operações de persistência da aplicação;
- **Prisma ORM:** atua como camada de comunicação entre a aplicação e o banco de dados PostgreSQL, simplificando consultas e manipulação dos dados.

Essa organização proporciona maior desacoplamento entre os componentes, reduz a duplicação de código e torna a aplicação mais preparada para futuras evoluções.

2.1 Stack Tecnológica

A implementação da API utiliza um conjunto de tecnologias consolidadas no desenvolvimento de aplicações back-end, selecionadas para atender aos requisitos funcionais e técnicos do projeto.

Tecnologia	Finalidade
Node.js 22	Ambiente de execução da aplicação.
Express 5	Framework responsável pelo gerenciamento das rotas e das requisições HTTP.
Prisma	Camada de persistência responsável pelo mapeamento entre objetos da aplicação e o banco de dados relacional.
PostgreSQL 15	Sistema Gerenciador de Banco de Dados utilizado para armazenar usuários e postagens.
Bcrypt	Criptografia das senhas armazenadas no banco de dados.
JWT (JSON Web Token)	Implementação da autenticação baseada em tokens.
Joi	Validação dos dados recebidos nas requisições da API.
Jest	Framework utilizado para testes automatizados.
Supertest	Biblioteca para testes de integração dos endpoints REST.
Swagger / OpenAPI	Documentação interativa dos endpoints da API.
Docker	Containerização da aplicação.
Docker Compose	Orquestração dos serviços necessários para execução do sistema.
Github	Controle de versão e gerenciamento colaborativo do projeto.
GitHub Actions	Automatização fluxos de trabalho de desenvolvimento de software diretamente no seu repositório GitHub.

2.2 Fluxo de Requisição

Toda requisição realizada à API percorre uma sequência de componentes até que a resposta seja retornada ao cliente. Esse fluxo garante que validações, autenticação e regras de negócio sejam executadas antes do acesso aos dados persistidos.

O fluxo da requisição pode ser representado da seguinte forma:



Durante esse processo:

- As **rotas** identificam qual endpoint foi solicitado;
- Os **middlewares** verificam se a requisição é válida e, quando necessário, autenticam o usuário e verificam suas permissões;
- Os **controllers** recebem a requisição e delegam sua execução aos serviços;
- Os **services** aplicam as regras de negócio da aplicação;
- Os **repositories** realizam as operações de persistência utilizando o Prisma ORM;
- O **PostgreSQL** armazena e recupera os dados solicitados;
- Por fim, a resposta é devolvida ao cliente no formato JSON.

2.3 Modelo de Dados

O domínio da aplicação foi modelado utilizando duas entidades principais: **User** e **Post**, que representam, respectivamente, os usuários da plataforma e as postagens publicadas.

A entidade **User** armazena as informações necessárias para autenticação e autorização dos usuários, incluindo e-mail, senha criptografada e perfil de acesso (Professor ou Aluno).

A entidade **Post** representa o conteúdo publicado na plataforma, contendo informações como título, conteúdo, data de criação e o identificador do professor responsável pela publicação.

As entidades são relacionadas por meio do atributo **authorId**, que estabelece uma relação entre cada postagem e seu respectivo autor. Dessa forma, cada postagem pertence a um único professor, enquanto um professor pode possuir diversas postagens cadastradas.

Essa modelagem garante integridade referencial dos dados e facilita futuras expansões da aplicação, como inclusão de comentários, categorias ou histórico de edições.

2.4 Segurança

A API implementa mecanismos de segurança para proteger os dados dos usuários e restringir o acesso às funcionalidades administrativas.

As senhas cadastradas nunca são armazenadas em texto puro. Antes de serem persistidas no banco de dados, passam por um processo de criptografia utilizando a biblioteca **bcrypt**, que gera um hash seguro para cada senha.

O processo de autenticação utiliza **JSON Web Token (JWT)**. Após realizar o login com credenciais válidas, o usuário recebe um token de acesso, que deve ser enviado em todas as requisições protegidas através do cabeçalho:

```
Authorization: Bearer <token>
```

Durante o processamento da requisição, um middleware valida a autenticidade do token e identifica o usuário autenticado.

Além da autenticação, a aplicação implementa um mecanismo de **autorização baseado em papéis (Role-Based Access Control – RBAC)**. Usuários com perfil **Professor** possuem permissão para criar, editar e excluir postagens, enquanto usuários com perfil **Aluno** têm acesso apenas às funcionalidades públicas de consulta.

Essa estratégia garante que apenas usuários autorizados possam modificar os conteúdos publicados na plataforma.

2.5 Endpoints da API

A API disponibiliza um conjunto de endpoints responsáveis pelo gerenciamento das postagens e pela autenticação dos usuários.

Os endpoints públicos permitem a consulta dos conteúdos disponíveis:

Método	Endpoint	Descrição
GET	/posts	Lista todas as postagens cadastradas.
GET	/posts/:id	Retorna uma postagem específica pelo identificador.
GET	/posts/search?q=termo	Realiza a busca de postagens por palavras-chave.

Os endpoints de autenticação permitem o cadastro e o acesso dos usuários:

Método	Endpoint	Descrição
POST	/auth/register	Realiza o cadastro de um novo usuário.
POST	/auth/login	Autentica o usuário e retorna um token JWT.

As operações de gerenciamento de conteúdo são protegidas e exigem autenticação:

Método	Endpoint	Permissão	Descrição
POST	/posts	Professor	Cria uma nova postagem.
PUT	/posts/:id	Professor	Atualiza uma postagem existente.
DELETE	/posts/:id	Professor	Remove uma postagem cadastrada.

3. Instalação e Configuração do Ambiente

Esta seção descreve o processo de preparação do ambiente de desenvolvimento necessário para execução da API REST do Blog Educacional. São apresentados os requisitos de software, a estrutura do projeto, a configuração das dependências, a inicialização do banco de dados, a execução da aplicação e os procedimentos para validação do ambiente.

A aplicação foi desenvolvida utilizando **Node.js** como ambiente de execução, **Express** para gerenciamento das requisições HTTP, **Prisma ORM** como camada de persistência e **PostgreSQL** como banco de dados relacional. Todo o ambiente pode ser executado localmente ou através de containers Docker, garantindo maior padronização entre os ambientes de desenvolvimento.

3.1 Pré-requisitos

Antes de iniciar a instalação do projeto, é necessário possuir as seguintes ferramentas instaladas na máquina:

Ferramenta	Versão	Finalidade
Node.js	20 ou superior	Execução da aplicação
npm	Compatível com Node.js	Gerenciamento das dependências
Git	Última versão	Clonagem do repositório
Docker	Última versão estável	Containerização da aplicação
Docker Compose	Última versão estável	Orquestração dos containers
PostgreSQL	15 ou superior	Persistência dos dados (quando executado localmente)

É recomendada a utilização de uma IDE como o **Visual Studio Code**, que oferece suporte ao ecossistema Node.js e facilita a execução, depuração e manutenção da aplicação.

3.2 Clonando o Repositório

O primeiro passo consiste em realizar o clone do repositório oficial do projeto.

```
git clone https://github.com/viniciusgodoy88/tech-challenge-blog2.git
```

Após o download, deve-se acessar o diretório da aplicação.

```
cd tech-challenge-blog2
```

3.3 Estrutura do Projeto

O projeto foi organizado seguindo uma arquitetura em camadas, permitindo maior organização e desacoplamento entre os componentes.

Uma visão simplificada da estrutura de diretórios é apresentada abaixo.

```
tech-challenge-blog/
├── prisma/
│   ├── migrations/
│   └── schema.prisma
├── src/
│   ├── controllers/
│   ├── middlewares/
│   ├── repositories/
│   ├── routes/
│   ├── services/
│   ├── validations/
│   ├── config/
│   └── server.js
├── tests/
├── Dockerfile
├── docker-compose.yml
├── package.json
└── README.md
```

Cada diretório possui uma responsabilidade específica:

Diretório	Responsabilidade
controllers/	Recebem as requisições HTTP e retornam as respostas ao cliente.
services/	Implementam as regras de negócio da aplicação.
repositories/	Centralizam o acesso ao banco de dados utilizando Prisma ORM.
routes/	Definem todos os endpoints da API.
middlewares/	Executam autenticação, autorização e validações.
validations/	Centralizam validações dos dados recebidos.
prisma/	Contém o schema do banco e as migrations.
tests/	Armazena os testes automatizados da aplicação.

Essa organização facilita a manutenção do código e permite que novos módulos sejam adicionados sem comprometer a arquitetura existente.

3.4 Instalação das Dependências

Após acessar o diretório do projeto, é necessário instalar todas as dependências definidas no arquivo **package.json**.

```
npm install
```

Esse comando realiza o download de todas as bibliotecas utilizadas pela aplicação, incluindo:

- Express;
- Prisma ORM;
- PostgreSQL Driver;
- JWT;
- bcrypt;
- Joi;
- Jest;
- Supertest;
- Swagger;
- demais dependências utilizadas pelo projeto.

Ao término da instalação, o diretório **node_modules** será criado automaticamente.

3.5 Configuração das Variáveis de Ambiente

A aplicação utiliza variáveis de ambiente para armazenar informações de configuração e dados sensíveis.

Essas configurações são definidas no arquivo **.env** localizado na raiz do projeto.

Um exemplo de configuração é apresentado abaixo.

```
DATABASE_URL="postgresql://usuario:senha@localhost:5432/blog"
JWT_SECRET="chave_secreta"
PORT=3000
NODE_ENV=development
```

As principais variáveis possuem as seguintes finalidades:

Variável	Descrição
DATABASE_URL	String de conexão com o PostgreSQL.
JWT_SECRET	Chave utilizada para assinatura dos tokens JWT.
PORT	Porta utilizada pela API.
NODE_ENV	Define o ambiente de execução (development ou production).

A utilização de variáveis de ambiente aumenta a segurança da aplicação e facilita sua implantação em diferentes ambientes.

3.6 Configuração do Banco de Dados

Após configurar as variáveis de ambiente, é necessário preparar o banco de dados.

Inicialmente, deve-se gerar o cliente do Prisma.

```
npx prisma generate
```

Em seguida, executam-se as migrations responsáveis pela criação das tabelas.

```
npx prisma migrate dev
```

Esse processo cria automaticamente toda a estrutura necessária para funcionamento da aplicação, incluindo as tabelas relacionadas aos usuários e às postagens.

Caso seja necessário visualizar os registros do banco de dados durante o desenvolvimento, o Prisma disponibiliza uma interface gráfica através do comando:

```
npx prisma studio
```

Essa ferramenta facilita a inspeção e manipulação dos dados armazenados durante os testes da aplicação.

3.7 Execução utilizando Docker

Para simplificar a configuração do ambiente e garantir maior padronização entre os desenvolvedores, a aplicação pode ser executada utilizando Docker Compose.

O comando abaixo realiza o build da aplicação e inicializa todos os containers configurados.

```
docker compose up --build
```

Durante esse processo são iniciados:

- servidor da aplicação;
- banco de dados PostgreSQL;
- demais serviços configurados no ambiente.

A utilização de containers reduz diferenças entre ambientes de desenvolvimento e produção, aumentando a confiabilidade da aplicação.

3.8 Inicialização da Aplicação

Com todas as dependências instaladas e o banco configurado, a API pode ser iniciada através do comando:

```
npm run dev
```

Ao ser iniciada, a aplicação executa as seguintes etapas:

1. Carrega as variáveis de ambiente.
2. Inicializa o servidor Node.js.
3. Configura o Express.
4. Registra todas as rotas da API.
5. Configura os middlewares.
6. Estabelece conexão com o banco de dados.
7. Aguarda requisições HTTP.

Após esse processo, a API estará pronta para atender requisições dos clientes.

4. Testes

A qualidade do software foi um dos pilares considerados durante o desenvolvimento da API REST do Blog Educacional. Com o objetivo de garantir o correto funcionamento das funcionalidades implementadas e atender aos requisitos estabelecidos pelo **Tech Challenge**, foi desenvolvida uma estratégia de testes automatizados abrangendo diferentes níveis da aplicação.

O projeto contempla **testes unitários**, **testes de integração** e **testes de ponta a ponta (End-to-End)**, permitindo validar desde as regras de negócio implementadas na camada de serviços até o fluxo completo de utilização da API.

A implementação dessa estratégia contribui para a identificação precoce de defeitos, reduz a ocorrência de regressões durante a evolução do sistema e aumenta a confiabilidade da aplicação.

Conforme especificado pelo Tech Challenge, a aplicação deveria atingir **pelo menos 20% de cobertura por testes unitários**. Ao final do desenvolvimento, esse requisito foi superado, alcançando os resultados esperados.

4.1 Testes Unitários

Os testes unitários foram desenvolvidos utilizando o framework **Jest**, com foco na validação das regras de negócio implementadas na camada de serviços da aplicação.

Durante esses testes, as dependências responsáveis pelo acesso ao banco de dados foram simuladas (*mocks*), permitindo validar exclusivamente a lógica de negócio sem a necessidade de conexão com o PostgreSQL.

Os principais cenários contemplados incluem:

- criação de postagens;
- validação de campos obrigatórios;
- listagem de postagens;
- busca de postagens por identificador;
- tratamento de registros inexistentes;
- busca por palavras-chave;
- validação de parâmetros obrigatórios.

A utilização de *mocks* nos repositórios permitiu isolar a camada de serviços, tornando os testes mais rápidos, previsíveis e independentes da infraestrutura da aplicação.

4.2 Testes de Integração

Os testes de integração tiveram como objetivo validar a comunicação entre os diferentes componentes da aplicação, verificando o funcionamento conjunto das rotas, controllers, services, repositories e banco de dados.

Esses testes foram implementados utilizando **Supertest**, responsável por simular requisições HTTP diretamente à API.

Entre os aspectos validados destacam-se:

- funcionamento dos endpoints REST;
- códigos de resposta HTTP;
- integração entre as camadas da aplicação;
- comunicação com o banco de dados;
- estrutura das respostas em formato JSON.

Essa abordagem assegura que os componentes da aplicação funcionem corretamente quando executados de forma integrada.

4.3 Testes de Ponta a Ponta (End-to-End)

Além dos testes unitários e de integração, foi implementado um teste de ponta a ponta (End-to-End), responsável por validar o fluxo completo de utilização da API.

PASS tests/integration/postRoutes.test.js					
PASS tests/e2e/authPostFlow.test.js					
PASS tests/unit/postService.test.js					
File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	64.03	66.66	29.41	71	
src	82.35	50	0	82.35	
app.js	82.35	50	0	82.35	14,26,35
src/controllers	59.37	66.66	40	61.29	
authController.js	83.33	66.66	75	88.23	12,34
postController.js	28.57	100	16.66	28.57	10-31
src/docs	100	100	100	100	
swagger.js	100	100	100	100	
src/repositories	22.72	0	10	29.41	
postRepository.js	22.72	0	10	29.41	11-36
src/routes	77.27	100	16.66	100	
authRoutes.js	100	100	100	100	
postRoutes.js	68.75	100	16.66	100	
src/services	82.35	91.66	66.66	86.66	
PostService.js	82.35	91.66	66.66	86.66	35-39

4.4 Ferramentas Utilizadas

A estratégia de testes foi implementada utilizando ferramentas consolidadas no ecossistema Node.js.

Ferramenta	Finalidade
Jest	Framework responsável pela execução e organização dos testes automatizados.
Supertest	Biblioteca utilizada para simulação de requisições HTTP durante os testes de integração e End-to-End.

A combinação dessas ferramentas permitiu automatizar a validação dos principais fluxos da aplicação de maneira simples e eficiente.

4.5 Execução dos Testes

A execução da suíte de testes é realizada através do comando:

```
npm test
```

Esse comando executa todos os testes automatizados do projeto e gera um relatório contendo a cobertura de código.

Ao término da execução, o Jest apresenta estatísticas referentes às principais métricas de cobertura (*Statements*, *Branches*, *Functions* e *Lines*), permitindo acompanhar a qualidade dos testes implementados.

Durante o desenvolvimento deste projeto, foram executadas **3 suítes de testes**, totalizando **11 casos de teste**, todos concluídos com sucesso, conforme demonstrado pelo relatório gerado pelo Jest.

4.6 Resultados Obtidos

Ao final da implementação da estratégia de testes, foram obtidos os seguintes resultados:

- **3 suítes de testes executadas;**
- **11 testes automatizados implementados;**
- **100% de sucesso na execução dos testes;**
- **64,03% de cobertura de Statements;**
- **66,66% de cobertura de Branches;**
- **29,41% de cobertura de Functions;**
- **71,00% de cobertura de Lines.**

Os testes implementados contemplam os principais fluxos da aplicação, incluindo validação das regras de negócio, comunicação entre as camadas da arquitetura e execução do fluxo completo de autenticação e criação de postagens.

5. Processo de Trabalho do Grupo

O desenvolvimento da API REST do Blog Educacional foi conduzido de forma colaborativa, mantendo a dinâmica de trabalho adotada na Fase 01 do Tech Challenge. A equipe organizou suas atividades por meio de reuniões síncronas periódicas para definição de prioridades, acompanhamento da evolução do projeto e discussão de decisões técnicas. Paralelamente, a comunicação assíncrona foi realizada por meio de um grupo no Discord, utilizado para esclarecimento de dúvidas, compartilhamento de informações e alinhamento entre os integrantes.

Para o gerenciamento das atividades, foi utilizado um quadro de tarefas, permitindo acompanhar o progresso de cada funcionalidade e organizar as entregas ao longo do desenvolvimento. O trabalho foi dividido em ciclos curtos, nos quais cada funcionalidade era implementada individualmente pelo responsável, validada por meio de testes e posteriormente revisada pelos demais membros antes de sua integração ao projeto principal.

Essa abordagem colaborativa contribuiu para reduzir conflitos durante o desenvolvimento, facilitar a identificação de problemas e manter a consistência da arquitetura e do código implementado.

5.1 Divisão de Responsabilidades

A distribuição das atividades buscou aproveitar as competências individuais de cada integrante, mantendo a colaboração entre todos durante as etapas de implementação, revisão e validação da solução.

Integrante	Responsabilidade Principal
Felipe B. Franco	Elaboração da documentação técnica, definição da arquitetura da solução e apoio na modelagem das regras de negócio.
Gabriel V. F. Sancinetti	Modelagem do banco de dados, definição do schema Prisma, implementação da autenticação e autorização utilizando JWT e integração com o PostgreSQL.
Vinicius Godoy	Desenvolvimento das rotas e controllers da API, containerização da aplicação com Docker e implementação dos testes automatizados.

Embora cada integrante tenha assumido responsabilidades específicas, as decisões técnicas foram discutidas em conjunto e validadas coletivamente durante o desenvolvimento.

6. Desafios e Experiências da Equipe

A migração de uma plataforma low-code, utilizada na Fase 01, para uma arquitetura baseada em **Node.js**, **Express** e **Prisma ORM** representou um aumento significativo na complexidade técnica do projeto. Diferentemente da solução anterior, a equipe passou a ser responsável por todas as camadas da aplicação, incluindo arquitetura, persistência de dados, autenticação, validações, containerização e testes automatizados.

Ao longo do desenvolvimento, diversos desafios contribuíram para o aprendizado prático dos conceitos apresentados durante o curso.

6.1 Padronização da Estrutura do Projeto

Durante o desenvolvimento, foram identificadas diferenças de comportamento entre sistemas operacionais quanto à sensibilidade entre letras maiúsculas e minúsculas nos caminhos dos arquivos. Enquanto alguns ambientes locais permitiam diferenças de capitalização sem gerar erros, os containers Linux utilizados pelo Docker exigiam correspondência exata entre os nomes dos arquivos e os caminhos de importação.

Essa experiência evidenciou a importância de manter uma padronização consistente da nomenclatura dos arquivos e realizar testes em ambientes semelhantes ao de produção.

6.2 Escolha da Tecnologia de Persistência

Durante a fase inicial do projeto foram avaliadas diferentes alternativas para persistência dos dados, incluindo soluções relacionais e não relacionais.

Após análise das características do domínio da aplicação, optou-se pela utilização do **PostgreSQL** em conjunto com o **Prisma ORM**, principalmente devido ao relacionamento existente entre usuários e postagens e à maior aderência do modelo relacional aos requisitos do sistema.

Essa decisão simplificou a modelagem dos dados e permitiu maior integração com os conceitos estudados durante a disciplina de Banco de Dados.

6.3 Gerenciamento de Dependências

Ao longo da implementação, diferentes bibliotecas foram avaliadas para atender às necessidades do projeto.

Como consequência, tornou-se necessário revisar cuidadosamente as dependências declaradas no projeto, removendo bibliotecas que deixaram de ser utilizadas e mantendo o ambiente consistente para todos os integrantes da equipe.

Esse processo contribuiu para reduzir conflitos de instalação e garantir maior estabilidade da aplicação.

6.4 Integração entre Autenticação e Autorização

Outro desafio relevante consistiu na implementação da autenticação utilizando **JSON Web Token (JWT)** integrada aos mecanismos de autorização da aplicação.

Foi necessário garantir que apenas usuários autenticados com perfil de **Professor** pudessem criar, editar ou excluir postagens, enquanto usuários com perfil de **Aluno** permanecessem com acesso apenas às funcionalidades públicas de consulta.

Essa implementação reforçou conceitos relacionados à segurança de APIs REST e ao controle de acesso baseado em papéis (*Role-Based Access Control – RBAC*).

6.5 Containerização da Aplicação

A utilização do Docker exigiu a configuração adequada dos containers responsáveis pela API e pelo banco de dados, bem como o entendimento do funcionamento da rede interna criada pelo Docker Compose.

Esse processo permitiu à equipe compreender aspectos importantes relacionados à padronização dos ambientes de desenvolvimento e implantação, reduzindo diferenças entre as máquinas utilizadas pelos integrantes.

6.6 Trabalho Colaborativo

Assim como na fase anterior, o desenvolvimento ocorreu de forma distribuída entre os integrantes da equipe.

A utilização do Git e do GitHub permitiu controlar as alterações realizadas no código, facilitar a integração das funcionalidades desenvolvidas individualmente e reduzir conflitos durante a evolução do projeto.

A comunicação contínua e a realização de revisões coletivas contribuíram para manter a consistência da arquitetura adotada e a qualidade da solução entregue.

7. Estado Atual do Projeto e Próximos Passos

Ao término desta fase, a aplicação atende aos requisitos funcionais estabelecidos pelo Tech Challenge, disponibilizando uma API REST para gerenciamento de postagens, autenticação de usuários, controle de acesso por perfis e persistência em banco de dados relacional.

Além das funcionalidades implementadas, foram identificadas oportunidades para evolução da solução em trabalhos futuros, entre elas:

- ampliação da cobertura de testes automatizados para atingir uma parcela ainda maior da aplicação;
- disponibilização da documentação da API por meio do Swagger em uma rota pública;
- implementação de uma pipeline de Integração Contínua (CI) utilizando GitHub Actions para execução automática dos testes e validações;
- inclusão de mecanismos de monitoramento, métricas e observabilidade da aplicação;
- expansão do domínio da aplicação com funcionalidades como comentários, categorias e gerenciamento de perfis de usuários.

Essas melhorias contribuem para aproximar a aplicação de um ambiente de produção e ampliar sua escalabilidade e facilidade de manutenção.

8. Considerações Finais

O desenvolvimento da API REST do Blog Educacional permitiu consolidar os conhecimentos adquiridos ao longo da disciplina, proporcionando à equipe experiência prática na construção de uma aplicação back-end utilizando tecnologias amplamente empregadas no mercado.

A migração de uma plataforma low-code para uma solução baseada em código proporcionou maior compreensão sobre conceitos fundamentais de Engenharia de Software, incluindo arquitetura em camadas, modelagem de dados, autenticação baseada em JWT, persistência utilizando Prisma ORM, containerização com Docker e implementação de testes automatizados.

Ao final do projeto, foram implementadas as funcionalidades previstas para a Fase 02, permitindo a consulta pública de postagens, o gerenciamento completo dos conteúdos por usuários autenticados com perfil de professor e a persistência das informações em banco de dados relacional.

Além do atendimento aos requisitos funcionais, o projeto incorporou práticas relacionadas à organização do código, documentação técnica, testes automatizados, versionamento e trabalho colaborativo, resultando em uma solução robusta, modular e preparada para futuras evoluções.

9. Documentos Extras

Link da apresentação no Youtube: https://youtu.be/Hr_pW55gvWc

Link do repositório: <https://github.com/viniciusgodoy88/tech-challenge-blog2>

10. Referências

- Docker. *Docker Documentation*. Disponível em: <https://docs.docker.com>. Acesso em: jun. 2026.
- Express.js. *Express — Node.js Web Application Framework*. Disponível em: <https://expressjs.com>. Acesso em: jun. 2026.
- FIAP. *Tech Challenge – Fase 02: Blog Educacional (API REST)*. Material didático. 2026.
- Jest. *Jest Documentation*. Disponível em: <https://jestjs.io>. Acesso em: jun. 2026.
- JWT.io. *Introduction to JSON Web Tokens*. Disponível em: <https://jwt.io/introduction>. Acesso em: jun. 2026.
- Prisma. *Prisma ORM Documentation*. Disponível em: <https://www.prisma.io/docs>. Acesso em: jun. 2026.
- Docker. *Docker Documentation*. Disponível em: <https://docs.docker.com>. Acesso em: jul. 2026.
- Express.js. *Express — Node.js Web Application Framework*. Disponível em: <https://expressjs.com>. Acesso em: jul. 2026.
- FIAP. *Tech Challenge – Fase 02: Blog Educacional (API REST)*. Material didático. São Paulo: FIAP, 2026.
- Git. *Git Documentation*. Disponível em: <https://git-scm.com/doc>. Acesso em: jul. 2026.
- GitHub. *GitHub Docs*. Disponível em: <https://docs.github.com>. Acesso em: jul. 2026.
- Google. *Gemini*. Disponível em: <https://gemini.google.com>. Acesso em: jul. 2026.
- Joi. *Joi Validation Library*. Disponível em: <https://joi.dev>. Acesso em: jul. 2026.
- Jest. *Jest Documentation*. Disponível em: <https://jestjs.io>. Acesso em: jul. 2026.
- Node.js. *Node.js Documentation*. Disponível em: <https://nodejs.org/docs/latest/api>. Acesso em: jul. 2026.
- OpenAI. *ChatGPT*. Disponível em: <https://chatgpt.com>. Acesso em: jul. 2026.
- PostgreSQL Global Development Group. *PostgreSQL Documentation*. Disponível em: <https://www.postgresql.org/docs>. Acesso em: jul. 2026.
- Prisma. *Prisma ORM Documentation*. Disponível em: <https://www.prisma.io/docs>. Acesso em: jul. 2026.
- Swagger. *OpenAPI Specification*. Disponível em: <https://swagger.io/specification>. Acesso em: jul. 2026.

- SuperTest. SuperTest Documentation. Disponível em: <https://github.com/ladjs/supertest>. Acesso em: jul. 2026.